D.4.2 Design Document

PlanER: Cost-Based Optimization of Hybrid Classical–LLM Entity Resolution Plans in a Document Database

Course: COMP 8157 Advanced Database Topics, Fall 2026 · **Authors:** Shashang Patel (patel3cd@uwindsor.ca), Nauman Syed (syed9i@uwindsor.ca) · School of Computer Science, University of Windsor · **Date:** October 28, 2026 · **Version:** 1.0

1. Introduction & Context

PlanER compiles one declarative entity resolution (ER) request into the cheapest MongoDB execution plan that meets a quality target or a budget. ER decides whether two records — for example “Apple iPhone 15 Pro 128GB Black” and “iPhone15 Pro (Blk) 128 GB” — describe the same real-world entity. Large language models (LLMs) match accurately but charge for every candidate pair they examine; classical matchers are nearly free but weaker on messy data. In current practice a developer picks one matcher and applies it to every pair, which either overpays (an LLM on clean bibliographic data) or loses quality (cheap rules on noisy product titles).

PlanER treats this as a query optimization problem. The user states *what* to match and the objective, e.g. `match abt with buy, budget $2` or `target F1 ≥ 0.85`. A cost-based optimizer then chooses the blocking index, the matcher cascade and the escalation thresholds, exactly as a SQL optimizer chooses access paths and join methods for a declarative query. The project therefore exercises four database principles: declarative abstraction, logical–physical data independence, cost-based optimization, and indexing as access-path selection.

In scope: declarative request handling; four blocking access paths (B-tree, multikey, `$search`, `$vectorSearch`); a five-tier matcher cascade (rules, Fellegi–Sunter, random forest, local LLM, hosted LLM); the optimizer (sampling, estimation, Pareto-pruned plan enumeration); execution and run logging; clustering with transitivity repair; a Streamlit demo; evaluation on DBLP–ACM, Abt–Buy, Amazon–Google, DBLP–Scholar and (stretch) WDC Products against LLM-only, best-classical and LOTUS-style embedding-proxy baselines.

Out of scope: production or multi-tenant deployment; a general query language beyond the fixed ER request; streaming re-resolution; engines other than MongoDB; enterprise authentication, auditing or role-based access control; a polished GUI beyond the demo.

Traceability: this design implements requirements FR-1 to FR-23 and NFR-1 to NFR-11 of the D.4.1 User Requirements and Analysis document, which in turn derive from the D.1 Pre-Proposal and D.2.1 Full Proposal. Requirement and acceptance-criterion IDs are cited inline throughout.

2. Goals & Non-Goals

The system succeeds when the optimizer’s chosen plan meets the user’s objective and its predictions match what actually happens at execution time.

Goal	Measurable success criterion	Req.
Meet the quality target	For θ = 0.85, observed F1 is within 0.05 of θ on at least 4 of 5 datasets	AC-1
High-recall blocking	Pair completeness ≥ 90% on DBLP–ACM and Abt–Buy for every access path; explain() logged per run	AC-2
Cut LLM spend	≥ 70% fewer LLM calls than the LLM-only baseline with a pairwise F1 drop ≤ 5 points	AC-3
Accurate optimizer	Predicted cost within 15% and predicted F1 within 0.05 of observed on at least 3 of 5 datasets	AC-4
Never pay twice	An identical re-run issues 0 new LLM calls (100% cache-hit rate)	AC-5, NFR-3
Respect the budget	Observed cost ≤ B in at least 95% of 20 trial runs; overruns logged with cause	AC-7, NFR-11
Fast estimation	Sample-based estimation < 5 min; the demo shows the plan, cost and F1 in < 5 s	NFR-1, AC-8
Scale	DBLP–Scholar (~168M possible pairs) blocked without exhaustive comparison	NFR-2, AC-10

Non-goals: beating state-of-the-art matching accuracy (Ditto-style fine-tuning is an optional tier only); a general-purpose semantic query language; real-time or incremental ER; production hardening, multi-user access control or high availability.

3. System Architecture

PlanER has five subsystems around a single MongoDB instance: a request layer, the optimizer, the execution engine (blocking, matcher cascade, clustering), the storage layer, and external model services (Figure 1).

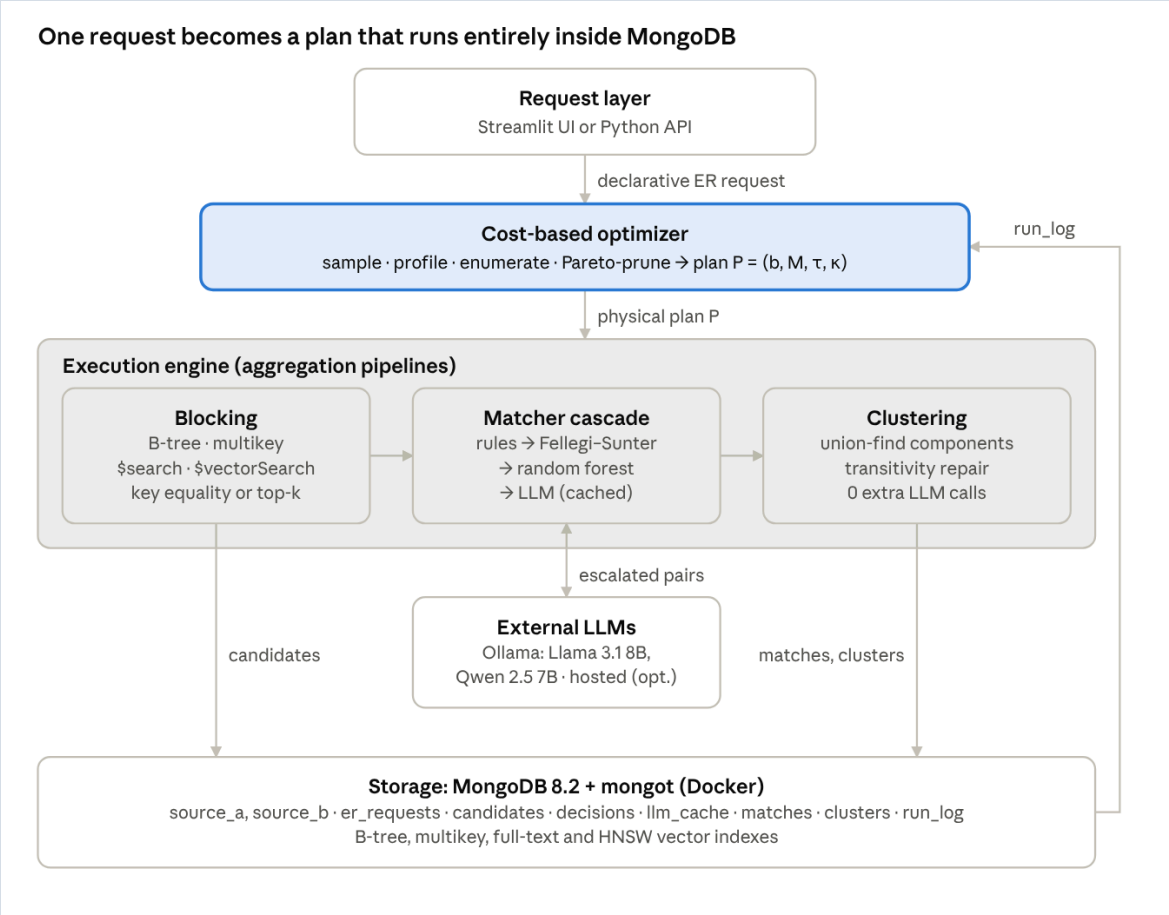


Figure 1. PlanER system architecture. Requests flow top to bottom; observed results in run_log flow back to the optimizer.

- **Request layer** (Streamlit UI or Python API) accepts the declarative request and stores it as an `er_requests` document (FR-1, FR-21).
- **Optimizer** draws a stratified labeled sample, profiles every access path and matcher, enumerates plans, prunes dominated ones and returns a physical plan P = (b, M, τ, κ) (FR-13 – FR-15).
- **Execution engine** runs P as aggregation pipelines: blocking writes `candidates`, the cascade writes `decisions`, clustering writes `matches` and `clusters` (FR-3 – FR-12, FR-16 – FR-18).
- **Storage layer** is MongoDB 8.2 Community Edition plus the `mongot` search process, which keeps full-text and vector indexes in sync through Change Streams.
- **External services:** Ollama serves the local LLMs; one optional hosted LLM API supplies real per-token prices (FR-10, FR-11).

Every executed plan writes observed cost, time and F1 to `run_log`, closing the loop so the optimizer's predictions can be compared against observations (FR-19, FR-20) — the same way real query optimizers are evaluated.

4. Logical Architecture

Every ER request has the same logical plan, **Normalize → Block → Match → Cluster**; only the physical operators change. This is logical–physical data independence: one request can run under many equivalent physical plans with the same meaning.

Logical operator	Input → output	Physical alternatives the optimizer chooses from
Normalize	raw record → enriched record	Fixed: lowercase, strip punctuation, token array, blocking key, 384-d sentence embedding (FR-2)
Block (b)	R, S → candidate pairs	B-tree key + `$lookup` · multikey token index · `$search` BM25 top-k · `$vectorSearch` HNSW top-k
Match (M, τ)	pair → match / non-match / escalate	rules → Fellegi–Sunter → random forest → local LLM → hosted LLM, each with thresholds (ℓ_i, u_i)
Cluster (κ)	matched pairs → entities	connected components · connected components + transitivity repair

The code is split into separate Python modules with documented interfaces (NFR-9): `ingest`, `blocking`, `matchers`, `optimizer`, `clustering` and `ui`. Every access path implements `candidates(req, k)` and every matcher implements `score(pairs)`, so the optimizer treats them as interchangeable operators and the text/vector paths can fall back to MongoDB Atlas without code changes (NFR-5).

5. Physical Architecture

The full stack runs in Docker Compose on each team member's laptop, so both environments are identical (NFR-10).

Container	Image / version	Role	Resources
`mongod`	MongoDB 8.2 Community	Collections, B-tree and multikey indexes, aggregation pipelines	Volume `/data/db`
`mongot`	MongoDB Search (public preview)	`$search` and `$vectorSearch` indexes, synced via Change Streams	Volume for index files
`ollama`	Ollama	Llama 3.1 8B and Qwen 2.5 7B with greedy decoding and a fixed prompt	GPU if present, else CPU
`planer-app`	Python 3.11	PyMongo, RapidFuzz, scikit-learn, sentence-transformers (all-MiniLM-L6-v2), Streamlit on port 8501	Host CPU
Fallback	MongoDB Atlas free tier	Same `$vectorSearch` stage if `mongot` is unstable	Cloud

The application container reaches `mongod` on port 27017 and Ollama on port 11434 over the Compose network; only Streamlit is exposed to the host. The optional hosted LLM and the optional Ditto tier (cloud GPU) are the only outbound calls.

6. Data Model

Records keep their original attributes plus derived fields, and every downstream document references record `_id`s that must exist in the source collections at run time (NFR-4). Figure 2 shows the entities and relationships; the table lists key fields and indexes.

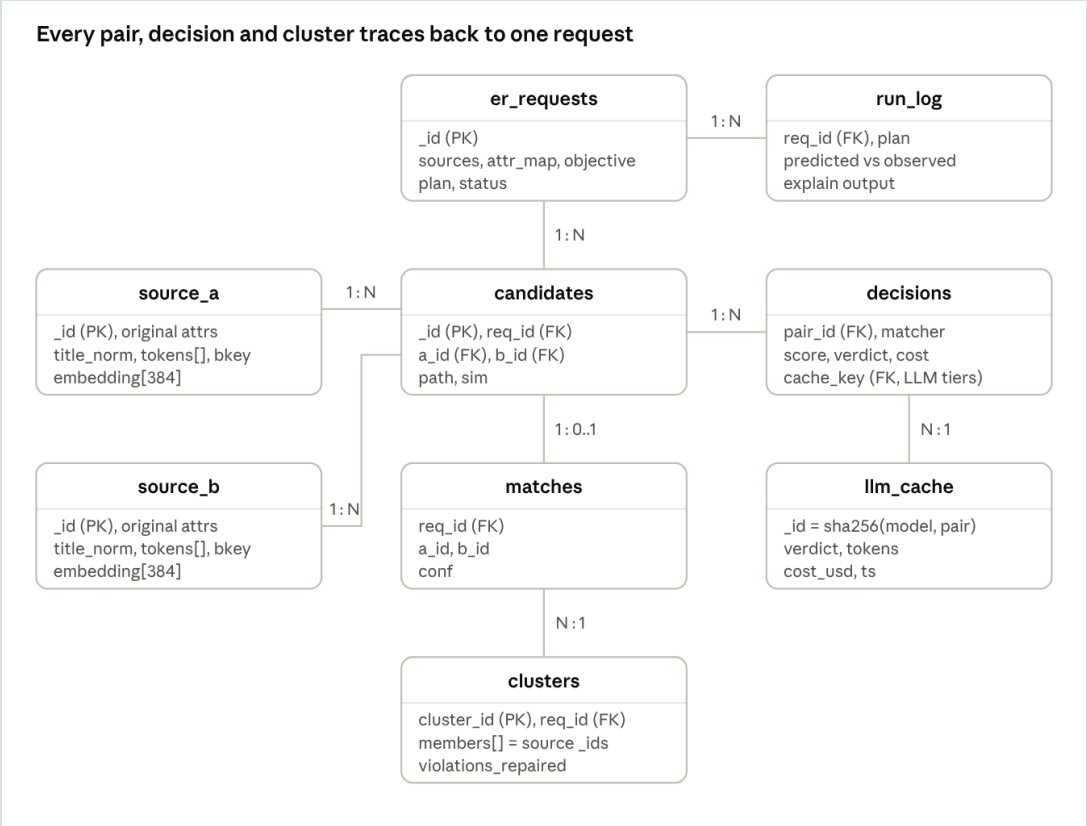


Figure 2. Entity-relationship diagram of the PlanER collections.

Collection	Key fields	Indexes
`source_a`, `source_b`	`_id`, original attributes (title, brand, price…), `title_norm`, `tokens[]`, `bkey`, `embedding[384]`	B-tree on `bkey`; multikey on `tokens`; `$search` on `title_norm`; `$vectorSearch` (HNSW, cosine) on `embedding`
`er_requests`	`_id`, `sources`, `attr_map`, `objective {type, value}`, `plan`, `status`	`_id`
`candidates`	`_id`, `req_id`, `a_id`, `b_id`, `path`, `sim`	Unique compound `{req_id, a_id, b_id}`
`decisions`	`pair_id`, `matcher`, `score`, `verdict`, `cost`, `cache_key`	`{pair_id, matcher}`
`llm_cache`	`_id = sha256(model, prompt_ver, a_id, b_id)`, `verdict`, `tokens`, `cost_usd`, `ts`	`_id` (point lookup)
`matches`, `clusters`	`req_id`, `a_id`, `b_id`, `conf` · `cluster_id`, `members[]`	`req_id`; multikey on `members`
`run_log`	`req_id`, `plan`, `pred {cost, f1}`, `obs {cost_usd, secs, llm_calls, f1_pair, f1_cluster, label_cost}`, `explain`	`{req_id, ts}`

Example record: `{ _id, title: "iPhone15 Pro (Blk) 128 GB", brand: "apple", title_norm: "iphone15 pro blk 128 gb", tokens: ["iphone15","pro","blk","128","gb"], bkey: "apple|iph", embedding: [384 floats] }`

Data flow: CSV → `mongoimport` → `source_*` (enriched by `ingest`) → blocking pipeline → `candidates` → cascade (reads/writes `llm_cache`) → `decisions` → `matches` → `clusters`; metrics from every stage → `run_log`. `llm_cache` acts as a materialized view of LLM results: it is checked before every model call and persists across requests (FR-12, NFR-3).

7. Component Design

The optimizer is the core algorithm; the other modules are operators it schedules. A plan's cost follows Eq. (1) of the D.2.1 proposal, where N_1 is the candidate count produced by access path b, c_i the per-pair cost of matcher m_i, and e_i the fraction of pairs m_i escalates:

$$C(P) = C_{block}(b) + \Sigma_{i=1..k}\, N_i \cdot c_i + C_{label}(P), \quad N_{i+1} = N_i \cdot e_i(\ell_i, u_i) \quad (1)$$

7.1 Optimizer (FR-13 – FR-15)

```
def optimize(req, sample_size=300):
    S = stratified_sample(req, sample_size, strata=sim_score_bins)  # keeps rare matches
    labels = label(S)                                   # labeling cost charged to C_label
    for b in ACCESS_PATHS: prof[b] = profile(b)         # N1, latency, pair completeness PC_b
    for m in MATCHERS:     prof[m] = run(m, S, labels)  # c_i; per (l,u): e_i, precision, recall
    plans = []
    for b in ACCESS_PATHS:
        for M in ordered_subsets(MATCHERS, keep_cost_order=True):
            for tau in threshold_grid(M):               # e.g. 5 x 5 grid per tier
                cost, f1 = estimate(b, M, tau, prof)    # Eq. (1); recall capped by PC_b
                plans.append(Plan(b, M, tau, cost, f1))
    pareto = [p for p in plans if not dominated(p, plans)]   # prune on (cost, F1)
    if req.objective.type == "theta":
        return min((p for p in pareto if p.f1 >= req.theta), key=lambda p: p.cost)
    return max((p for p in pareto if p.cost <= req.budget), key=lambda p: p.f1)
```

7.2 Matcher cascade (FR-7 – FR-12)

```
def cascade(pairs, M, tau):
    pending = pairs
    for i, m in enumerate(M):
        last = (i == len(M) - 1)
        if m.is_llm:                                    # never pay twice
            cached = llm_cache.find({"_id": {"$in": [key(m, p) for p in pending]}})
            record(cached); pending = pending - cached
        s = m.score(pending)
        for p in pending:
            if s[p] >= tau[i].u or (last and s[p] >= 0.5): decide(p, MATCH)
            elif s[p] <= tau[i].l or last:                decide(p, NON_MATCH)
        if m.is_llm: llm_cache.insert_many(new_verdicts)
        pending = [p for p in pending if tau[i].l < s[p] < tau[i].u]
    return decisions
```

7.3 Blocking (FR-3 – FR-6). Each access path is one aggregation pipeline. The B-tree path uses `$lookup` on `bkey` (e.g. brand + first three title characters); the multikey path uses `$unwind` on tokens and `$lookup` on rare tokens (document frequency below a cut-off); the text and vector paths run `$search` / `$vectorSearch` with `limit: k` per record. For each path we record pair completeness, reduction ratio, time and index size, and store `explain("executionStats")` in `run_log` (FR-16).

7.4 Fellegi–Sunter (FR-8). Comparison vectors are discretized similarity levels per mapped attribute; m- and u-probabilities are fitted by expectation–maximization without labels; the match weight $\Sigma \log(m_k/u_k)$ is calibrated to a score in [0, 1]. It is implemented directly in Python over pairs fetched from MongoDB.

7.5 Clustering (FR-17, FR-18). Union-find builds connected components from `matches`. Inside a component, any pair the cascade decided as a confident non-match (score ≤ ℓ) is a transitivity violation (A–B and B–C matched, A–C not). Repair removes the lowest-confidence edge on the path between the two records and re-splits the component, with zero additional LLM calls.

8. Interface Design

Users reach PlanER through one Streamlit page or one Python call; both submit the same JSON request. Moving the budget or θ re-selects a plan from the cached Pareto set, so the plan, cost and F1 update in under 5 s (AC-8, NFR-8).

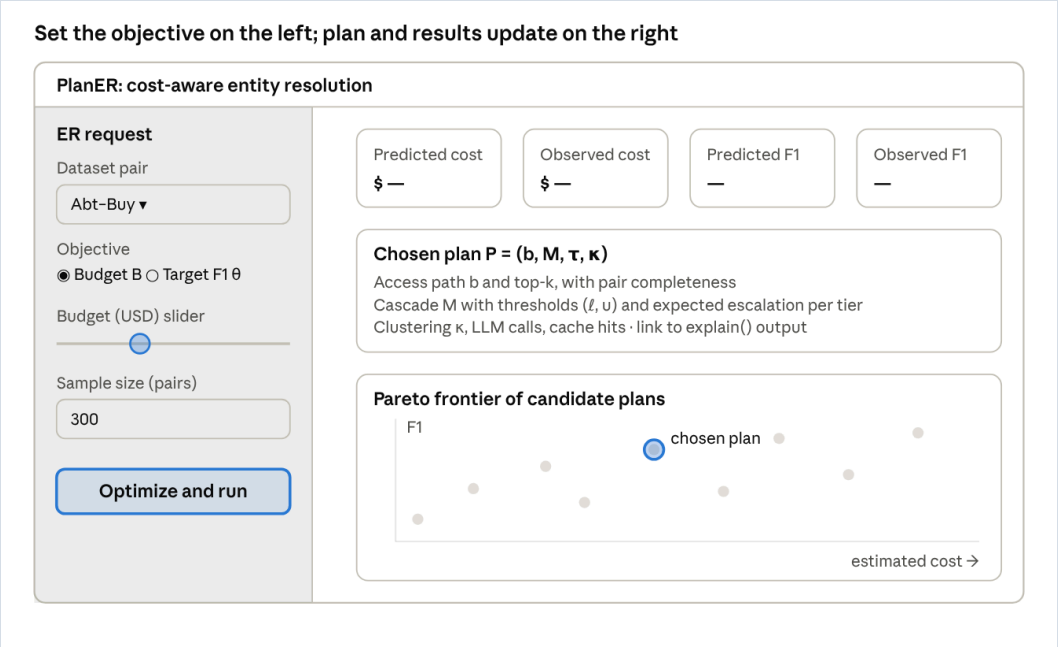


Figure 3. Streamlit demo wireframe; the highlighted point is the selected plan on the Pareto frontier.

Request document (stored in `er_requests`):

```
{ "sources":   ["abt", "buy"],
  "attr_map":  {"title": "name", "brand": "manufacturer", "price": "price"},
  "objective": {"type": "budget", "value": 2.00},      // or {"type": "theta", "value": 0.85}
  "sample_size": 300 }
```

Internal API call (Python, NFR-9)	Returns
`planer.submit(request) -> req_id`	Stores the request and returns its id
`planer.optimize(req_id) -> Plan`	Chosen P = (b, M, τ, κ), the Pareto set, predicted cost and F1
`planer.execute(req_id, plan) -> RunReport`	Observed cost, time, LLM calls, pairwise and cluster F1
`BlockingPath.candidates(req, k)`	Iterator of `(a_id, b_id, sim)`
`Matcher.score(pairs)`	Calibrated scores in [0, 1] plus per-pair cost
Ollama `POST /api/generate`	`{"verdict": "match" | "no_match"}` from a fixed prompt

Constraints & Security

Area	Constraint	Design response
Hardware	Laptop CPU/GPU; 7–8B LLMs are slow per pair	Only escalated pairs reach an LLM; verdict cache; fall back to smaller models if needed
Bottleneck	DBLP–Scholar has ~168M possible pairs	Key-equality or top-k blocking; DBLP–Scholar used mainly for blocking experiments (NFR-2)
Bottleneck	`mongot` index build and memory for 64K × 384-d vectors	Indexes built once at load time; Atlas free-tier fallback (NFR-5)
Stability	`$search`/`$vectorSearch` are public preview	Common access-path interface; development and evaluation use only
Reproducibility	LLM output can vary; benchmark contamination	Greedy decoding, fixed prompt, disjoint train/estimation/test splits (NFR-7); WDC Products limits contamination
Security	API keys and database credentials	Environment variables or a local secrets file; never committed or written to `run_log` (NFR-6)

Security	Network exposure	MongoDB authentication enabled, bound to the Compose network; only port 8501 exposed
Cost	Budget B must hold	Optimizer applies a safety margin; any overrun is logged with its cause (NFR-11)

9. Design Rationale

Each choice keeps ER inside the database and turns the cost–quality trade-off into something an optimizer can reason about.

Choice	Why	Alternative rejected
MongoDB 8.2	One engine offers B-tree, multikey, full-text and vector indexes, so blocking becomes access-path selection inside the database	PostgreSQL + pgvector / FAISS (pre-proposal): vector search outside the engine
Enumeration + Pareto pruning	The plan space (4 paths × cost-ordered subsets × threshold grid) is small and interpretable, in the spirit of Selinger-style access-path selection	Reinforcement-learning policy (CaRL-EM): needs training, opaque
Stratified sample (~300 pairs)	Estimates e_i, cost and F1 cheaply while keeping rare matches; labeling cost is part of the objective	Full labeling: defeats the cost goal
Heterogeneous cost-ordered cascade	Rules and Fellegi–Sunter decide most easy pairs at near-zero cost; only ambiguous pairs reach an LLM	LLM-only, or LLM-ladder cascades (FrugalGPT, LOTUS)
Own Fellegi–Sunter	Short to implement and runs on pairs fetched from MongoDB	Splink: does not run on MongoDB
Ollama: Llama 3.1 8B, Qwen 2.5 7B	Free, local, deterministic with greedy decoding	Hosted-only LLMs: cost and rate limits
all-MiniLM-L6-v2 (384-d)	Small, fast on CPU, compact HNSW index	Larger embedding models: memory and index size
Hash-keyed `llm_cache`	Materialized results guarantee zero repeat LLM calls	Recomputing verdicts on every run
Union-find + repair	Near-linear time; repair needs no model calls	Correlation clustering: costly, may need extra calls
Streamlit + Docker	Fast demo UI; identical environments for both members	Custom web front end: out of scope

10. Revision History

Version	Date	Author(s)	Description of changes	Status
v0.1	2026-10-26	S. Patel, N. Syed	Initial architecture, logical/physical views and data model derived from D.2.1 and D.4.1.	Draft
v0.2	2026-10-27	S. Patel, N. Syed	Added component pseudocode, interface design, constraints & security and design rationale.	Draft
v1.0	2026-10-28	S. Patel, N. Syed	Added diagrams (architecture, ERD, wireframe) and requirement traceability; finalized as D.4.2.	Submitted
v1.1	pending	pending	Reserved for instructor/TA feedback and resulting revisions.	Pending approval